Computer Science E214

Practical Test 3

2025

Instructions

1. Put your student card face-up on the desk, clearly visible, during the test.
2. Submit your solutions on SunLearn before the time allocated for the test has expired. Submissions will close automatically at the end of the test.
3. You must complete the practical test and submit your solutions using the lab computers in the lab allocated for the tutorial.
4. This is a closed-book test: Unless explicitly specified otherwise, you may not access internet resources, your textbook, previous tutorials or memos, or any other notes during the test. You may also not communicate with anyone about the test except the course staff. You may only have the following applications/windows open on your computer during the test:
   - Webpages from STEMLearn, the Python documentation at `docs.python.org`, the PowerShell documentation at `https://learn.microsoft.com/en-us/powershell/`, and the vim help pages at `vimhelp.org` as the only open tabs in your browser—STEMLearn may only be used for accessing the test and making submissions for it;
   - the test instructions—if they are issued in a digital format;
   - one or more vim/gvim editor windows for any code files or other text files you are writing/editing/viewing;
   - output windows generated by your code during execution;
   - one or more file manager/explorer and/or terminal/console windows for performing tasks related to the test (such as running and testing your code); and
   - additional windows as required to view any other resources issued as part of the test.

   You may therefore not have any other text editors or text file viewers except vim/gvim open, including any integrated development environments (IDEs) such as VSCode. You must also be logged out of all other university services (e.g. Outlook) and other accounts (such as your Google account).
5. Having any other windows open or being logged into such other services is considered cheating, and is grounds for awarding zero for the practical test and taking disciplinary steps for cheating.
6. Test conditions apply in the venue until all students have finished writing. If you leave the venue before the end of the test, you may not log on to SunLearn or communicate in any way with students still in the venue until the test is finished.
7. Please monitor the projected screens in the venue for any additional announcements during the test. We recommend checking at least every 10 minutes whether an update has been placed there. We use the screens to avoid disturbing students unduly with announcements.

I Only Want to Hear One Click, Redux

For this question, you will work towards implementing a tool for efficiently guiding someone through debugging all possible configurations of a system specified by a number of binary switches, where the status of each switch's setting (on or off) is indicated by a light on an instrument panel. The tool should (a) read information on the various switches from standard input; (b) show (on standard output) an ordered sequence of configurations to test—beginning with all switches off—such that changing from each configuration to the next only involves changing the state of a single switch; and (c) use `stddraw` to display the state of the instrument panel at each step, highlighting the light corresponding to the last switch toggled.

Illustrative example

Consider running the following command, where the `$` denotes the command prompt:

```
$ python classy_oneclick.py 2000 < switches.txt
```

In this example, the command-line argument of 2000 indicates a 2000ms delay between successive steps in the output, while the file `switches.txt` contains information on the number of switches, and the location, size, and colour of each switch on the instrument panel. Suppose the content of `switches.txt` is as follows:

```
3
0 0 255 0.2 0.5 0.1
0 255 0 0.5 0.5 0.1
255 0 0 0.8 0.5 0.1
```

The first line indicates the number of switches as an `int`, and each subsequent line provides information about one switch, with whitespace-separated fields:

- The first three fields are `int` values between 0 and 255 (inclusive), specifying the red, green, and blue components of the switch's indicator light colour respectively. These values can be provided to the `Color` constructor to obtain a `Color` instance specifying the indicator light's colour.
- The next two fields are `float` values between 0 and 1, indicating the x- and y-coordinates of the centre of the switch's indicator light.
- The last field is a `float` specifying the radius of the switch's indicator light.

The expected output of your program in the terminal in this case should be as follows[1]:

```
Off Off Off
On Off Off
On On Off
Off On Off
Off On On
On On On
On Off On
Off Off On
```

[1]Note that importing `stddraw` often leads to an additional two lines of output to the terminal, although this may be suppressed in your configuration. It does not matter whether these additional output lines appear or not, as long as they do not appear on standard output after any of the switch configurations.

Along with this output, the animation `classy_oneclick_2000_switches.gif`—available for download from the practical test activity on STEMLearn—should be displayed in a `stddraw` window. Each frame displays the indicator lights for the switches that are on at that time, with a white circle highlighting the indicator light for the switch that was toggled from the previous step (if any). Note that each line of output on the standard output stream should only be printed when the corresponding frame of the animation begins.

Code skeleton

You are provided with a code skeleton called `classy_oneclick.py`. To complete the test, you must suitably complete the eight functions and/or methods whose bodies currently only contain a docstring and the `pass` statement. The purpose/behaviour of these functions/methods is documented in the relevant docstrings, with additional comments in the file also being potentially helpful. **Important**: you MUST NOT change the program in any way except by completing the bodies of these functions/methods (and potentially removing certain `#type: ignore` comments as indicated in the code skeleton). If you do so, you may be awarded a mark of zero, or be heavily penalized.

Detailed specifications

Most details on what is required are provided in the docstrings for the missing functions and/or methods in the code skeleton. Below, we provide some additional comments which may be helpful to address potential confusion/ambiguity.

- Assume your program will be called with a valid number of command-line arguments, and all command-line arguments specify positive integers.
- Assume your functions will only be called with arguments of the types specified in the relevant signatures during testing; in turn, your functions should only return values of the specified return types. You can check for potential issues using `mypy`.
- Your program should output a number of lines to standard output (i.e. in the terminal), each representing a distinct configuration of the switches, and with each configuration after the first the same as the previous one except for the status of a single switch. Your output must begin with all switches off, and you must use recursion to find the sequence of configurations.
- **The order of the switches in each line of output must match the order the switches are read in from standard input, and the order that switches are toggled for the first time must also match this order.**
- After outputting each line, the indicator panel for the new configuration should be displayed in `stddraw`, highlighting the indicator light (whether off or on) of the switch that was toggled from the previous frame. Note that for the first frame, there should be no highlighting.
- This frame should be displayed for the time specified by the second command-line argument.
- The next line of output should only be generated after this frame has been displayed for the required time.

Approach

You may be awarded marks for completing various functions and/or methods individually, even if the other functions/methods do not work, as long as your code compiles successfully. Some additional comments on some of these functions follow:

- `def plot(setting: list[Switch], highlight: int, delay: int) -> None:`

  This function should be called with suitable arguments from `animate_switches`, and should use the `plot` method of the `Switch` class to plot the individual switches with `stddraw`, using the default x- and y-scales and canvas size. Assume that indicator lights do not overlap, and are all in the unit square $[0, 1] \times [0, 1]$.

- `def oneclick(n: int) -> list[int]:`

  This is the function that generates and returns the sequence of switch changes required for `n` switches. Note the comment in bold under "Detailed specifications" above about the order that switches must be toggled for the first time.

  You **MUST** implement this function recursively. The key idea for this recursive implementation is that if one knows how to generate a sequence of switch changes for K switches, one can run through these changes with an additional switch added at the end[2] initially set to off, covering all configurations of $K + 1$ switches where the last switch is off. One can then switch the last switch on, leaving the remaining K switches in the same configuration. The rest of the switch changes, with the last switch now on, can then be performed by going through the original sequence of switch changes for K switches in reverse order. One approach for the base case corresponds to zero switches, where there are clearly no switch changes necessary.

Mark allocation and marking notes

- You will receive a mark of zero if your code does not compile successfully with `python -m py_compile classy_oneclick.py`.
- You may receive a mark of zero, or be heavily penalized, if you make any changes to the code skeleton besides completing the specified function/method bodies, and removing certain occurrences of `#type: ignore` as noted in the relevant method docstrings.
- You will receive zero for any function that violates encapsulation by directly accessing or modifying fields of the `Switch` class, rather than making use of the `Switch` API implicitly specified by the code skeleton. (This obviously does not apply to methods implemented in the `Switch` class.)
- While their implementations differ significantly in complexity, each function/method is equally weighted for the final test mark—except that the function and the method for which you achieve the highest mark will count double the other functions/methods.
- For most functions/methods, you will get a mark of zero or one, depending on whether it works as specified or not on various test cases. For some more involved functions/methods, partial marks may be awarded in the case of partially correct behaviour.

Submission

Submit a single Python program file named `classy_oneclick.py`. Your submitted file will be renamed if its name is not exactly correct.

[2] Other placements of this switch are possible; however, for this test, you must effectively place the additional switch at the end.